

Documentation

Overview

This system enables real-time remote control of JetBot robots over a network. A user on a PC can discover, connect to, and send commands to a robot, while receiving live sensor data back. The design prioritizes simplicity, low latency, and correctness for a local or same-network deployment. The architecture went through several design iterations and protocol evaluations before settling on the current approach.

Architectures Considered

First, the different protocols that can be used in problems are listed below.

Data Transmission & Communication Protocols

1. MQTT. In this architecture each node the robot, player, and user has its own pub/sub broker, and the brokers connect to each other. Each node publishes to its own broker and subscribes to the others. This gives good scalability and clustering support but this is not a direct p2p between user robot and player. It gives guaranteed delivery, easy cloud storage integration, and the broker detects disconnections instantly. It was not chosen as the primary protocol because it requires a standalone broker process such as Mosquitto or EMQX running separately, adding operational overhead for a local deployment.
2. WebRTC-Based Distributed Mesh. In this architecture a signaling server runs in the cloud and WebRTC handles all three connections, Robot to User, Robot to Player, and User to Player. Each node connects to the other two and data flows directly peer to peer. This is not a pub,sub system and It has a single point of failure if the cloud fails.
3. ROS2 based pub-sub is the closest but the issue is it should be on the same network for it to work. Hence, not used in this system.
4. Kafka is more used for logging and not immediate pub sub systems.
5. gRPC streaming is less flexible for multiple pub/sub patterns.
6. NATS was noted as a strong alternative, particularly for multi-robot deployments at scale. It was not pursued because it introduces another external dependency which is not a direct p2p.

ZMQ was chosen because it matches the specification exactly, independent pub/sub servers spawned per robot with no separate broker needed. The player process itself is the broker. It gives low latency on LAN, measured at 2–6ms in testing, and is simple to implement and debug. As stated in the reference documentation: ZeroMQ is the perfect match for this exact specification because other technologies require brokers or don't spawn independent pub/sub servers per entity.

The key tradeoff is that ZMQ has no built-in guaranteed delivery and no strict message ordering. For this system that is acceptable, sensor data is high frequency so occasional drops are not critical, commands are short, and there is no requirement for duplication prevention. The system makes no strict ordering or delivery guarantees by design.

Why Not Pure WebRTC

WebRTC was seriously considered for the full control path but rejected for different reasons. First mainly it's not a direct pub sub system like zmq. WebRTC can silently disconnect without the robot knowing the operator is gone. P2P fails on some networks causing the operator to lose control completely. There is no guaranteed delivery, which matters for safety-critical commands where an ACK is needed. I was considering a MQTT for commands and webrtc just for video streaming etc. a hybrid approach to work well. But this hybrid approach is more broker based pub sub and if indirect communication is allowed this would be better for Prod level robots. Mainly because for safety critical systems there must be a message guarantee to avoid issues.

User

```
|— MQTT → cloud broker → robot (commands: move, stop, configure)
|— WebRTC → P2P direct → robot (video stream + high-freq sensor data)
```

Why Not Loop Over Sessions for ZMQ

An alternative considered was looping over sessions to run ZMQ for multiple robots from a single process. This was removed because it would open multiple sockets to the same proxy, cause duplicate messages, and add complexity with no benefit. The per-robot player process model is cleaner and provides true isolation.

Player Per Robot Design

Each robot gets its own dedicated player process. This gives isolation so one robot's traffic never affects another. Each player binds its own ports so scaling is independent. Routing is simple because the player only handles one robot's topics with no filtering complexity needed. The cloud tracks the player PID and sends SIGTERM on deregister or timeout.

Topic Structure

Topics are namespaced by robot ID in the format robot/{robot_id}/sensor, robot/{robot_id}/command, robot/{robot_id}/status, and robot/{robot_id}/processed. This means multiple robots can run simultaneously without topic collisions. Extending to multiple robots from the user side is straightforward, take multiple robot IDs as input and subscribe to all their topics. As demonstrated in testing, the system ran over 25 robots simultaneously with independent heartbeats and sensor streams.

Things that didn't work

ZMQ CURVE encryption was implemented initially but caused silent message drops because the robot and user generate fresh key pairs each run without pre-exchanging them with the player. CURVE silently drops all unverified messages with no error, making it extremely difficult to debug. CURVE was removed and replaced with token-based security described below.

I tried using a redis to store robots rather than locally but connectivity and it interfered with spawned players so removed it. I realized this is very cloud dependent and can be changed to sql or amazon s3 etc mostly based on what cloud the prod system uses.

Security

Two layers of security are implemented without using ZMQ CURVE which was found to silently break message delivery. At the HTTP layer all /connect requests require a user-token header matching the ZMQ_TOKEN defined in the shared config. Anyone without the correct token gets a 403 before they ever receive port numbers, so they cannot attempt a ZMQ connection at all. This was verified in testing, an unauthorized user gets the response "Not authorized to connect to the robot" and cannot proceed.

At the ZMQ layer every message carries a token field that the player checks on receipt. Messages with a wrong or missing token are silently dropped. This prevents unauthorized publishers from injecting commands or spoofing sensor data even if they discover the ZMQ port numbers by other means. Further improvements on security is needed based on what cloud we use and more control on access between robot user and player.

Multi-network access

For cross-network deployment across different locations three options were evaluated. The simplest is a free public MQTT broker such as broker.hivemq.com which requires zero setup and works immediately but is public with no topic privacy. The most realistic production option is a self-hosted broker giving full control and the closest experience to real production. The cleanest option for private cross-network communication is Tailscale VPN which makes machines on different networks behave as if they are on the same LAN with no open ports needed and completely private routing.

Latency

Latency is measured end-to-end by stamping every sensor message with a timestamp at the robot and computing the difference at the user on receipt. In testing on a local network this measured consistently between 2ms and 6ms, well within the 50ms target for responsive teleoperation and far below the 150ms threshold where control starts feeling unnatural.

Edge Cases Handled

Duplicate robot registration returns the existing ports rather than spawning a second player, verified in testing where connecting the same robot ID from two terminals shared one registration and allowed both to see user commands, which was useful for debugging. Player spawn failure returns a 500 with the stderr output. Robot not found returns a 404 on heartbeat, connect, and deregister. Bad or malformed ZMQ messages are caught and logged without crashing the player loop. Register retries up to 3 times with exponential backoff if the cloud is temporarily unreachable.

Assumptions and Known Limitations

Message delivery is best-effort with no strict ordering guarantee and no deduplication. If the robot disconnects mid-command it continues its last movement until a stop command

arrives, there is no automatic stop on disconnect at this stage. The main ambiguity I had was choosing between direct P2P and indirect communication. I ultimately stuck with ZMQ because it works well for local warehouse-based robots, and the question clearly states a P2P mesh.

If no heartbeat is received for 30 seconds, the robot is marked offline and the player is terminated. Until then, reconnection is reattempted, giving the robot time to recover in case of network issues. Multiple users can subscribe to the same robot's topics simultaneously since ZMQ pub/sub is broadcast-based; however, command conflicts between multiple users are not resolved, the last write is the final state.

The system is designed primarily for same-network or LAN deployment. The main areas for improvement are guaranteeing message delivery and resolving user command conflicts.

SOME OUTPUTS:

```
[Player] RECEIVED [robot/r8/sensor]: {'robot_id': 'r8', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.123953}
[Cloud] Heartbeat from r6
INFO: 127.0.0.1:55726 - "POST /heartbeat/r6 HTTP/1.1" 200 OK
[Player] RECEIVED [robot/r4/sensor]: {'robot_id': 'r4', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.162176}
[Player] RECEIVED [robot/r23/sensor]: {'robot_id': 'r23', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.3102238}
[Player] RECEIVED [robot/r12/sensor]: {'robot_id': 'r12', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.3101602}
[Player] RECEIVED [robot/r16/sensor]: {'robot_id': 'r16', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.310236}
[Cloud] Heartbeat from r25
INFO: 127.0.0.1:55727 - "POST /heartbeat/r25 HTTP/1.1" 200 OK
[Player] RECEIVED [robot/r5/sensor]: {'robot_id': 'r5', 'state': {'state': 'forward', 'speed': 0.5}, 'timestamp': 1775072560.3844097}
[Cloud] Heartbeat from r7
INFO: 127.0.0.1:55728 - "POST /heartbeat/r7 HTTP/1.1" 200 OK
[Player] RECEIVED [robot/r24/sensor]: {'robot_id': 'r24', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.4986665}
[Player] RECEIVED [robot/r20/sensor]: {'robot_id': 'r20', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.4986842}
[Player] RECEIVED [robot/r13/sensor]: {'robot_id': 'r13', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.4986649}
[Player] RECEIVED [robot/r9/sensor]: {'robot_id': 'r9', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.4994621}
[Player] RECEIVED [robot/r6/sensor]: {'robot_id': 'r6', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.5301518}
[Player] RECEIVED [robot/r17/sensor]: {'robot_id': 'r17', 'state': {'state': 'stopped', 'speed': 0.5}, 'timestamp': 1775072560.5299888}
```

```
PS C:\Users\Anandhini\Desktop\Robotics\Robot-system-Design\Robot-system-Design> python .\user\u2.py
List of Robots: ['r2', 'r1', 'r3', 'r4', 'r5', 'r6', 'r7', 'r8', 'r9', 'r10', 'r11', 'r12', 'r13', 'r14', 'r15', 'r16', 'r17', 'r18', 'r19', 'r20', 'r21', 'r22', 'r23', 'r24', 'r25']
Enter robot id: r5
C:\Users\Anandhini\AppData\Roaming\Python\Python313\site-packages\zmq\future.py:718: RuntimeWarning: Proactor event loop does not implement add_reader family of methods required for zmq. Registering an additional selector thread for add_reader support via tornado. Use `asyncio.set_event_loop_policy(WindowsSelectorEventLoopPolicy())` to avoid this warning.
  self.get_loop()
Command: RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072549.277505}
latency: 5.1ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072550.2923954}
latency: 3.5ms
foRECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072551.3091948}
latency: 4.4ms
rwardRECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072552.3118134}
latency: 3.2ms

[User] SENT: forward
Command: RECEIVED [robot/r5/command]: {'command': 'forward', 'speed': 0.5}
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'forward', 'speed': 0.5}, 'timestamp': 1775072553.3209124}
latency: 2.8ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'forward', 'speed': 0.5}, 'timestamp': 1775072554.3289135}
latency: 4.8ms
```

```

RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072546.2604508}
latency: 3.2ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072547.2683864}
latency: 6.3ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072548.2768528}
latency: 3.0ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072549.2775605}
latency: 4.5ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072550.2923954}
latency: 3.9ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072551.3091948}
latency: 4.4ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'backward', 'speed': 0.5}, 'timestamp': 1775072552.3118134}
latency: 3.4ms
RECEIVED [robot/r5/command]: {'command': 'forward', 'speed': 0.5}
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'forward', 'speed': 0.5}, 'timestamp': 1775072553.3209124}
latency: 2.8ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'forward', 'speed': 0.5}, 'timestamp': 1775072554.3289135}
latency: 4.7ms
RECEIVED [robot/r5/processed]: {'robot_id': 'r5', 'state': {'state': 'forward', 'speed': 0.5}, 'timestamp': 1775072555.3348897}
latency: 2.7ms

```

FOR NOT AUTHORIZED USER.

PS C:\Users\Anandhini\Desktop\Robotics\Robot-system-Design\Robot-system-Design>

python .\user\user2.py

List of Robots: ['r17']

Enter robot id: r17

[User] Connect response: [{'error': 'Not authorized to connect to robot'}, 404]

```

## Security

- **HTTP** - all `/connect` requests require a `user-token` header
  matching `ZMQ_TOKEN`
- **ZMQ** - every message includes a token field, rejected by the
  player if wrong
- Unauthorized users cannot get port info or publish/subscribe to
  any topic

---

## Latency Monitoring

The user client automatically prints latency for every received
sensor message:
```
latency: 12.3ms
```

This is measured as the time between the robot sending the message
and the user receiving it. Useful for diagnosing network issues.

```

References

1. <https://github.com/KeplerC/awesome-cloud-robotics>

2. <https://medium.com/@jiayi.hoffman/live-video-streaming-of-security-robots-28ffaeceaed4>
3. <https://medium.com/@jiayi.hoffman/control-security-robots-over-the-internet-f14451b06543>
4. [A Cloud-based Robot System for Long-term Interaction: Principles, Implementation, Lessons Learned](#)
5. <https://gist.github.com/hmartiro/85b89858d2c12ae1a0f9>
6. <https://github.com/aljanabim/nats-ros-connector>
7. <https://oneuptime.com/blog/post/2026-02-02-nats-pubsub-implementation/view>
8. Chatgpt for debugging errors mainly and understanding some networking concepts.